SIMATS ENGINEERING
CO5-ASSESSMENT TOOL 3

COURSE NAME: DIGITAL CIRCUITS COURSE CODE: ECA02

COURSE OUTCOME COVERED

CO:5 Analyze the operation and characteristics of NMOS and PMOS transistors, CMOS inverters, and MOS device parameters used in digital circuit design (BTL 3 & 4)

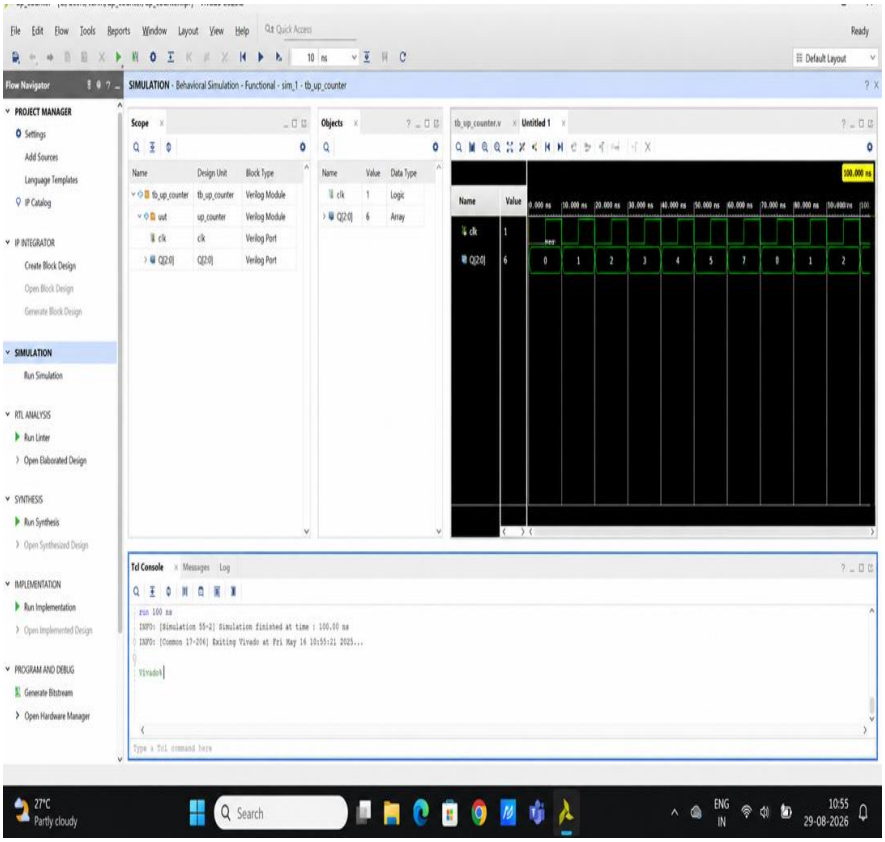
Assessment Tool number : AT3
Assessment Tool Weightage : 40%
Assessment Tool Name : HDL Code Error Debugging
Duration of this assessment : 90 Minutes
Marks : 20
Type of Assessment conduction : Inside Class

Description:

- Each student will be assigned one HDL Code Error Debugging problem 20 marks inside class. Analyze and debug one faulty analog circuit.
- Simulate the corrected circuit using VIVADO and verify results.
- Document the debugging process, observations, and conclusions.
- Upload VIVADO files, simulation results, and report to GitHub.
- Evaluation is based on fault identification, debugging approach, simulation accuracy, documentation, and GitHub submission.

Name: Sujaydayaalan C
Reg No:192412067

Item	Student Entry
1. Given Code	**3-Bit Up Counter** module up_counter(input clk, output reg [2:0] Q); always @(posedge clk) begin Q = Q + 1; end endmodule **Tasks** 1. Identify the coding issue. **2.** Rewrite using appropriate assignment statements.
2. Expected code output	A 3-bit register (Q) that increments by 1 on every positive edge of the clock, counting 000 → 001 → 010 → ... → 111 → 000 (wraps around), synthesized as sequential logic (flip-flops).
3. Observed Fault / Symptom	The code uses a **blocking assignment (=)** inside a clocked (always @(posedge clk)) block instead of a **non-blocking assignment (<=)**. In simulation this may appear to "work" for this simple single-statement case, but it will: • Cause simulation/synthesis mismatch in more complex designs • Violate standard RTL coding guidelines for sequential logic • Potentially cause race conditions if Q is used elsewhere in the same or another always block
6. Error Identified	**Incorrect assignment operator type used in sequential (clocked) logic.** Blocking assignment (Q = Q + 1;) used instead of non-blocking assignment (Q <= Q + 1;) inside an edge-triggered always block. **Rule violated:** For sequential logic (anything inside always @(posedge clk)), always use **non-blocking assignments**

	(<=). Blocking assignments (=) should only be used for **combinational logic** (always @(*)).
7. Correction Made	Changed the blocking assignment operator = to the non-blocking assignment operator <= in the statement Q = Q + 1; → Q <= Q + 1;
8. Corrected HDL code	module up_counter(input clk, output reg [2:0] Q); always @(posedge clk) begin Q <= Q + 1; end endmodule
9. Verification / Simulation Output	
10. Final Result & Technical Justification	**Final Result:** The corrected code produces a functionally correct, synthesizable 3-bit up counter. The output Q increments sequentially on every rising edge of clk and wraps around

after reaching its maximum value (7 → 0), matching the expected behavior of an up counter.

Technical Justification:

Aspect	Blocking (=) — Original	Non-Blocking (<=) — Corrected
Execution	Updates immediately, in sequential program order	Updates scheduled and applied at end of time step
Intended use	Combinational logic (always @(*))	Sequential logic (always @(posedge clk))
Hardware mapping	Can cause simulation mismatch with actual flip-flop behavior	Correctly models flip-flop (edge-triggered register) behavior
Risk if misused	Race conditions, incorrect values when signal is read elsewhere in same clock edge	None — matches real hardware timing

Using <= ensures that all right-hand-side values in the always block are evaluated using the *old* values of Q before any updates occur — exactly how physical flip-flops behave on a clock edge. This guarantees the RTL code's simulation behavior matches its synthesized hardware behavior, which is essential for reliable digital design.

Conclusion: The student's original logic (increment on clock edge) was conceptually correct, but the assignment operator choice violated standard RTL coding practice for sequential circuits. Replacing = with <= fixes the issue while preserving the intended counting function.